



Discrete Optimization

Average-weight-controlled bin-oriented heuristics for the one-dimensional bin-packing problem[☆]Krzysztof Fleszar^{a,*}, Christoforos Charalambous^b^a Olayan School of Business, American University of Beirut (AUB), P.O. Box 11-0236, Riad El Solh, Beirut 1107 2020, Lebanon^b Department of Computer Science, Frederick University, P.O. Box 24729, 1303 Nicosia, Cyprus

ARTICLE INFO

Article history:

Received 9 March 2010

Accepted 2 November 2010

Available online 5 November 2010

Keywords:

Packing

Heuristics

Bin-packing

Bin-oriented heuristics

Reductions

ABSTRACT

Bin-oriented heuristics for one-dimensional bin-packing problem construct solutions by packing one bin at a time. Several such heuristics consider two or more subsets for each bin and pack the one with the largest total weight. These heuristics sometimes generate poor solutions, due to a tendency to use many small items early in the process. To address this problem, we propose a method of controlling the average weight of items packed by bin-oriented heuristics. Constructive heuristics and an improvement heuristic based on this approach are introduced. Additionally, reduction methods for bin-oriented heuristics are presented. The results of an extensive computational study show that: (1) controlling average weight significantly improves solutions and reduces computation time of bin-oriented heuristics; (2) reduction methods improve solutions and processing times of some bin-oriented heuristics; and (3) the new improvement heuristic outperforms all other known complex heuristics, in terms of both average solution quality and computation time.

© 2010 Elsevier B.V. All rights reserved.

1. Introduction

Given an unlimited number of bins with an integer capacity $c > 0$ each, a set of n items, $N = \{1, \dots, n\}$, and an integer weight w_i , $0 < w_i \leq c$, for each item $i \in N$, the off-line one-dimensional bin-packing problem (BPP) is to assign each item to one bin, such that the total weight of items in each bin does not exceed c and the number of bins used is minimized. The solution using m bins is defined by a partition of N into m subsets, each corresponding to the contents of one bin.

The BPP is known to be NP-hard in the strong sense (Garey and Johnson, 1979). Its importance stems from the fact that it has vast industrial applications and it also frequently occurs as a subproblem of various practical problems.

The literature on the BPP is substantial. A standard reference for the BPP and a number of related problems is a book by Martello and Toth (1990). For BPP, they describe a number of simple heuristics and lower bounds, introduce a reduction procedure, MTRP, and an exact algorithm, MTP. A number of algorithms is studied by Wäscher and Gau (1996). A hybrid grouping genetic algorithm is presented by Falkenauer (1996). Approximation algorithms and

their performance ratios are investigated by Coffman et al. (1997). Scholl et al. (1997) offer a survey and introduce an exact algorithm, BISON. Schwerin and Wäscher (1997) present an improved version of the MTP of Martello and Toth (1990), using a lower bound derived from the cutting stock problem. Valério de Carvalho (1999) offers an exact algorithm using column generation and branch-and-bound. Vanderbeck (1999) presents an exact algorithm based on column generation for the cutting stock problem and shows that it performs well on some types of BPP instances. Gupta and Ho (1999) introduce a minimum-bin-slack (MBS) enumerative heuristic, which constructs BPP solutions by filling one bin at a time, each as much as possible. Fleszar and Hindi (2002) describe an improved implementation of MBS, introduce the MBS' heuristic, which fixes the largest remaining item in a bin before proceeding with enumeration, and offer several MBS-based heuristics, as well as a variable-neighbourhood-search meta-heuristic. Worst-case performance of heuristics like MBS and MBS', which proceed by solving a subset-sum problem for one bin at a time, is studied by Caprara and Pferschy (2004, 2005). Fekete and Schepers (2001) present a class of lower bounds for BPP, based on dual-feasible functions. Valério de Carvalho (2002) studies LP models for bin-packing and cutting stock problems. Alvim et al. (2004) present a highly effective hybrid improvement heuristic. Bhatia and Basu (2004) offer a multi-chromosomal grouping genetic algorithm and a better fit heuristic. Singh and Gupta (2007) propose a compound heuristic, combining a hybrid steady-state grouping genetic algorithm and an improved version of the Perturbation MBS' of Fleszar and Hindi (2002). Evolutionary algorithms

[☆] This work was partially supported by PLPRO/0506/02 Research Grant titled "Novel Technologies for Resource Optimization in the Glass Cutting Industry" funded by the Cyprus Research Promotion Foundation.

* Corresponding author.

E-mail addresses: kfleszar@gmail.com (K. Fleszar), c.charalambous@frederick.ac.cy (C. Charalambous).

are studied by Poli et al. (2007) and Rohlfshagen and Bullinaria (2007). Crainic et al. (2007a,b) introduce fast lower bounds and study their worst-case performance. Finally, Loh et al. (2008) present a weight annealing heuristic.

Most heuristics for BPP begin by sorting items such that their weights are in non-increasing order. Two well-known fast heuristics, the first-fit-decreasing (FFD) and the best-fit-decreasing (BFD) (see Martello and Toth, 1990), construct solutions in an item-oriented fashion: while unpacked items remain, the largest is packed in one of the bins: FFD packs it in the first bin in which it fits and BFD in the bin with the smallest but sufficient residual capacity. A new bin is added every time an item that does not fit in any of the initiated bins is encountered. Trivial implementations of both FFD and BFD take $O(n^2)$ time, but both heuristics can be programmed using sophisticated data structures to have $O(n \log n)$ time complexity.

This paper is concerned with the bin-oriented heuristics (BOHs) that construct solutions of the BPP by packing one bin at a time. Section 2 defines and provides several examples of such heuristics. The major contribution of this work is the new sufficient average weight (SAW) principle, described in Section 3, that can be used to improve both solutions and processing times of BOHs that consider multiple subsets of items for each bin. The new heuristics created with the help of the SAW principle are presented in Section 3. Section 4 shows how reduction methods can be used not only before but also within BOHs. Section 5 shows how all the ideas presented in this paper can be combined to define a competitive improvement heuristic. Section 6 presents an extensive computational study that testifies to the efficiency and effectiveness of all presented approaches. Finally, conclusions and further research directions are discussed in Section 7.

Throughout the paper, the following terms are used. A subset of items is *feasible* if its total weight does not exceed the bin capacity. A subset of items is *maximal* if it is feasible but adding any of the currently unpacked items would make it infeasible.

2. Bin-oriented heuristics

A bin-oriented heuristic (BOH) for the BPP constructs a solution as follows: while unpacked items remain, add a bin and pack in it a maximal subset of unpacked items. Consider the following examples of BOHs with their respective single-bin packing procedures:

- The first-fit-decreasing (FFD) heuristic (Martello and Toth, 1990) implemented in a bin-oriented manner packs each bin by adding to it the largest item that fits until no more items do. Implemented in this way, the computational complexity of FFD is $O(n^2)$.
- The best-two-fit (B2F) heuristic considers two maximal subsets of items for each bin and packs the one that leaves the smaller slack. The first subset is the same as the FFD subset. The second is created if the FFD subset has the total weight less than bin capacity and the smallest item in this subset can be replaced with a pair of even smaller items without exceeding bin capacity. If such a replacement is not possible, the first subset is adopted. Otherwise, the second subset is created by modifying the first, as follows: the smallest item is replaced with two other items of maximum total weight that fit; if as a result the load is smaller than before, an attempt is made to add other items, largest first, until no more items fit. Note that this definition of B2F differs from the one described by Friesen and Langston (1991), but both have a computational complexity of $O(n^2)$.
- The minimum-bin-slack (MBS) heuristic of Gupta and Ho (1999) considers for each bin all maximal subsets of unpacked items and packs the subset that leaves the smallest slack. If ties occur, preference is given to the subset with lexicographically

larger items. The computational complexity of MBS is $O(2^n)$, but if the maximum number of items that fit in one bin is u , it can be reduced to $O(n^{u+1})$.

- The MBS' heuristic of Fleszar and Hindi (2002) constructs a solution in the same way as MBS, except that for each bin it considers only those subsets that contain the largest currently unpacked item. The computational complexity of MBS' is $O(2^n)$, but if the maximum number of items that fit in one bin is u , it can be reduced to $O(n^u)$.

The difference between the four above BOHs is in the extent of search performed to determine a subset for each bin: FFD constructs only one maximal subset, B2F considers at most two, MBS' considers all maximal subsets that include the largest unpacked item, and MBS considers all maximal subsets.

Subset selection procedures used in MBS and MBS' can be programmed as a depth-first enumeration of all feasible subsets, in which items are considered in the order of non-increasing weights (see the Appendix for details). Alternatively, both heuristics can be implemented using dynamic programming approach, in which case their computational complexity is pseudo-polynomial, $O(n^2c)$. Both approaches are very time consuming in the worst case. The depth-first enumeration is often more practical, since it can find very good solutions even if early termination after a predefined number of iterations is used.

3. SAW principle and SAW heuristics

Regardless of the extent of search performed by a BOH, the goal of subset selection is to maximize the load or, equivalently, minimize the slack in the currently packed bin. Thus, when a bin is packed, the problem of packing subsequent bins is ignored. In difficult problem instances, BOHs such as B2F, MBS, and MBS' may use many relatively small items while packing the initial bins, leaving only relatively large items for packing in later stages. Since large items can seldom be combined efficiently, the last few bins may have large slacks and the solutions may be poor.

To avoid the above problem, we propose a mechanism of controlling the average weight of items packed in bins, based on a principle that we call the *sufficient average weight* (SAW) principle. A subset of items has a *sufficient average weight* if the average weight of its items is at least as large as the average weight of all currently unpacked items (including items in the subset). A BOH is said to follow the SAW principle if it selects one of the maximal subsets that have sufficient average weight or, if no such subset exists, the maximal subset that violates the sufficiency condition the least.

Based on the SAW principle, we introduce three new heuristics: SAWB2F, SAWMBS, and SAWMBS'. These differ from their non-SAW counterparts only in the objective used to select a subset for each bin: if maximal subsets with sufficient average weight exist among subsets considered by a heuristic, the one leaving minimum slack is selected; otherwise, the maximal subset violating the sufficiency condition the least is selected.

Note that the SAW principle can only be applied to those BOHs that consider more than one subset of items for each bin. Hence, FFD cannot benefit from it.

Consider a BPP instance with bin capacity equal to 15 and 8 items with weights (7, 6, 6, 6, 5, 5, 4, 4), for which the optimal solution uses three bins. FFD packs the items in four bins: (7, 6), (6, 6), (5, 5, 4), and (4) (numbers denote weights of packed items). B2F packs the first bin differently, replacing the item of weight 6 with two items of weight 4 each. Then, no three items can share a bin. As a result, a different but also 4-bin solution is found: (7, 4, 4), (6, 6), (6, 5), and (5). Although MBS and MBS' consider more subsets than B2F for each bin, they both find the same solution as B2F. Note that FFD fails to

find the 3-bin solution, because it does not attempt to improve the loads in the bins, while B2F, MBS, and MBS' fail because they use the two smallest items to improve the load in the first bin without considering the difficulty of packing the remaining items.

Consider now how the SAW heuristics solve the same example. The SAW principle asks that the subset packed in the first bin have an average weight at least as large as the average weight of all items, which is equal to $(7 + 6 + 6 + 6 + 5 + 5 + 4 + 4)/8 = 43/8 = 5.375$. The two subsets considered by SAWB2F, (7, 6) and (7, 4, 4), have average weights equal to 6.5 and 5, respectively. Only the first is sufficiently large, so the first subset, (7, 6), is packed in the first bin. Then, the average weight of the remaining items is $(6 + 6 + 5 + 5 + 4 + 4)/6 = 30/6 = 5$. The two subsets considered by SAWB2F are now (6, 6) and (6, 5, 4). Their average weights, 6 and 5, respectively, are both sufficiently large. Therefore, the subset leaving the smaller slack, (6, 5, 4), is now packed. The third bin is packed with the three remaining items, and as a result the 3-bin optimal solution is found. While SAWMBS and SAWMBS' consider more subsets than SAWB2F for some bins, they both find the same solution as SAWB2F.

In the above example, the SAW principle affects the subset selection for the first bin, preventing BOHs from packing in it the two smallest items, but does not restrict the subset selection for the subsequent bins. This illustrates the adaptive nature of the SAW principle: for the first bin, the average weight required is 5.375, which effectively prevents packing 3-item subsets in the first bin; after two larger than average items are packed in the first bin, the average weight required when packing the second bin is lowered to 5, allowing selecting a 3-item subset. The SAW principle can also be viewed as a lookahead mechanism that prevents packing in the bin at hand such subsets that would make the residual packing problem too difficult.

It is important for a heuristic following the SAW principle to select subsets only among maximal subsets. Consider another BPP instance with bin capacity equal to 7 and 5 items with weights (5, 5, 5, 2, 1). The SAW principle calls for a subset for the first bin to have the average weight at least as large as $(5 + 5 + 5 + 2 + 1)/5 = 18/5 = 3.6$. If a heuristic is allowed to accept non-maximal subsets, then (5) will be packed in the first bin instead of (5, 2) or (5, 1), because the average weight of (5) is sufficient ($5 \geq 3.6$), while the average weights of the latter two subsets are not sufficient ($3.5 \not\geq 3.6$ and $3 \not\geq 3.6$, respectively). While this decision will not prevent the heuristic from finding an optimal solution in this example, it is in general prudent not to leave unused space in a bin if there is an item that would fit in it. If only maximal subsets, (5, 2) and (5, 1), are considered by a heuristic, neither has sufficiently large average weight. In this case, the SAW principle calls for selecting the subset that violates the sufficiency condition the least. Thus, (5, 2) will be packed in the first bin, (5, 1) in the second, and (5) in the third.

As will be shown in the computational analysis, the SAW principle is capable of not only significantly improving solutions of BOHs, but also of significantly reducing computation time of those BOHs that enumerate a large number of feasible subsets for each bin, such as MBS and MBS'. This is because the minimum average weight requirement can also be used to prune significant parts of the enumeration tree generating subsets of items for one bin. Let $L = (1, \dots, |L|)$ be the list of currently unpacked items renumbered such that their weights are non-increasing. Assume that feasible subsets of items are enumerated in a depth-first manner, considering items in the order of list L . Consider a stage in the enumeration tree in which a partial subset of items, $A \subseteq \{1, \dots, j-1\}$ ($1 \leq j \leq |L|$), is selected, and items $\{j, \dots, |L|\}$ are still to be considered for addition to A . Assume that A^* is the current incumbent subset, $w(A)$ denotes the total weight of items in A , $\bar{w}(A)$ denotes the average weight of items in A , and $s(A)$ denotes the slack in a bin after packing A in it. Pruning of the enumeration tree is based on two requirements on the generated subset:

Maximality requirement: Since item j is the largest of the items to be considered for addition to A , at least $p_{\min} = \min\{[s(A)/w_j], |L| - j + 1\}$ items must be added to A for it to become maximal. Thus, the average weight of any maximal subset generated in the subtree will be at most $\bar{w}_{\max} = (w(A) + p_{\min} w_j) / (|A| + p_{\min})$.

Sufficient average weight requirement: If a generated subset is to replace the incumbent, A^* , its average weight must be at least as large as $\min\{\bar{w}(L), \bar{w}(A^*)\}$. Since the total load will never exceed c , the cardinality of a generated subset must be at most $\lfloor c / \min\{\bar{w}(L), \bar{w}(A^*)\} \rfloor$, so at most $p_{\max} = \lfloor c / \min\{\bar{w}(L), \bar{w}(A^*)\} \rfloor - |A|$ items can be added to A . Thus, the slack of the generated subset will not be less than $s_{\min} = \max\{s(A) - p_{\max} w_j, 0\}$.

If $p_{\min} > p_{\max}$, the two requirements contradict each other, so the subtree can be pruned. If the incumbent has sufficiently large average weight ($\bar{w}(A^*) \geq \bar{w}(L)$) and $\bar{w}_{\max}$ is not sufficiently large ($\bar{w}_{\max} < \bar{w}(L)$) or $\bar{w}_{\max}$ is sufficiently large ($\bar{w}_{\max} \geq \bar{w}(L)$) but the slack cannot be improved ($s_{\min} \geq s(A^*)$), then the subtree can be pruned. If the incumbent does not have sufficiently large average weight ($\bar{w}(A^*) < \bar{w}(L)$) and $\bar{w}_{\max}$ is even worse ($\bar{w}_{\max} < \bar{w}(A^*)$) or $\bar{w}_{\max}$ is as good as the average weight of the incumbent ($\bar{w}_{\max} = \bar{w}(A^*)$) but the slack cannot be improved ($s_{\min} \geq s(A^*)$), then the subtree can also be pruned.

A more detailed description of SAWMBS is provided in the Appendix. In particular, the pseudo-code of the enumerative procedure used within SAWMBS is provided, including the pruning algorithm described above.

4. Reduction methods

Reduction methods for BPP simplify BPP instances by packing some items in bins in such a way that optimal solutions can still be achieved. Such methods can be used before any algorithm for BPP is started. In BOHs, when a bin is filled, the remaining problem is still a BPP, but defined on a smaller set of items. Hence, we propose to use reduction methods after packing each bin. The rationale is that when some items are packed by a BOH, additional reductions may become possible.

Martello and Toth (1990) introduced a reduction method, called MTRP, which identifies undominated maximal subsets of at most three items each, and packs these subsets in separate bins. A subset is undominated if packing it in a bin does not prevent a possibility of finding an optimal solution. For detailed definition of dominance see (Martello and Toth, 1990). The computational complexity of MTRP is $O(n^2)$.

Preliminary experiments showed that calling MTRP before starting BOHs is often beneficial and does not increase computation time significantly. However, calling MTRP repeatedly after packing each bin is too time consuming. Therefore, in addition to using MTRP initially, we use the following simpler reduction methods after packing each bin. All but the last of these reductions are subsumed by MTRP. All have a computational complexity of $O(n)$, assuming unpacked items are sorted.

- If the largest unpacked item cannot share a bin with the smallest unpacked item, then a new bin with the largest item packed in it can be added to the solution.
- If the largest unpacked item can share a bin with the smallest unpacked item but not with the two smallest unpacked items, then a new bin can be added to the solution and two items can be packed in it: the largest unpacked item and the largest of the remaining unpacked items that fits.
- If the total weight of unpacked items is less than or equal to bin capacity, then a new bin with all remaining items packed in it can be added to the solution and the BOH can be terminated.

Note that one more $O(n)$ reduction can be defined: if two items exist that fill a bin completely, then they can be packed in a bin together. However, using this reduction is not necessary, since after the initial run of MTRP this reduction will never apply.

5. Perturbation-SAWMBS heuristic

Perturbation-MBS' is an improvement method introduced first by Fleszar and Hindi (2002) that successively builds a new solution by perturbing the current one. In each perturbation, an item from a bin with a relatively large slack is selected as a seed and a subset containing this item, considering all other items, is created employing the MBS' subset selection procedure. Items of the new subset are transferred to a new bin, which is added to the solution. This perturbation is repeated a number of times. Bins that become empty during this process are removed from the solution. The best solution encountered is saved as an incumbent and returned as the final solution.

Fleszar and Hindi (2002) defined perturbation such that the chance of improving the perturbed solution is maximized. To achieve this, the seed and the item order are defined such that the larger the slack of the bin in which an item resides, the higher the chance of using the item by the perturbation. Thus, the seed is selected randomly from all items with probability of choosing item i proportional to s_i , where s_i denotes the slack in the bin in which item i is currently packed. The remaining items are considered by MBS' in the order of non-increasing values of s_i , breaking ties arbitrarily.

A modified Perturbation-MBS' was suggested by Singh and Gupta (2007) who used it as part of their genetic algorithm for BPP. Three changes were proposed. First, bins filled completely by two items were never affected. Secondly, a step after each perturbation was added that repacked items from bins affected by the perturbation using the best-fit-decreasing heuristic. Finally, the new solution replaced the current only if the number of occupied bins did not increase.

We propose a new heuristic, which we call Perturbation-SAWMBS, which extends the Perturbation-MBS' in a different way. First, observe that if the total slack of the incumbent is less than c , then the incumbent is optimal and the heuristic should be terminated. Thus, while the perturbations are performed, it can be assumed that the total slack of the incumbent is greater than or equal to c . Perturbation-SAWMBS performs a new perturbation, described below, as long as the total slack in the incumbent is greater than c . If the total slack in the incumbent is equal to c , then the Perturbation-SAWMBS reverts to the old perturbation of the Perturbation-MBS'. The reason for switching to the old perturbation is that it works much better than the new one for those instances for which optimal solution has zero slack in all bins, and such a solution is required to improve an incumbent that has a total slack equal to c . Regardless of which perturbation is used, the newly created solution replaces the previous one.

The new perturbation orders all items randomly, choosing the first item, then the second, and so on, until all items are ordered, each time with a probability of choosing item i proportional to $w_i \times (s_i + 1)$. Such probability gives higher preference to larger items as well as to those items that reside in bins with larger slacks. When all items are ordered, a subset of items that can fit in one bin is determined using the first-fit approach. The items of this subset are transferred to a new bin, which is added to the solution. Then, all items residing in bins with non-zero slack are removed from their bins, except for the items just moved to the new bin, and all empty bins are deleted. Thereafter, all possible 2-for-1 exchanges between the packed set and the unpacked set of items are made: two items packed in one bin are exchanged with one unpacked item of the same weight as their combined

weight. The rationale is that such exchanges do not affect the loads in packed bins, but make the residual problem defined on the unpacked set of items easier, since it will involve smaller items. When such exchanges are no longer possible, the unpacked items are packed in bins using the SAWMBS heuristic with simple reductions, which completes the new perturbation.

The Perturbation-SAWMBS heuristic is used as follows. First, run MTRP and remove the items packed by MTRP from further consideration. For the reduced problem, compute the linear relaxation lower bound, $LB = \lceil \frac{1}{c} \sum w_i \rceil$. This lower bound is used to terminate the heuristic as soon as a solution with LB bins is found. Then, pack the items of the reduced problem with FFD, SAWB2F, and SAWMBS, each using the simple reductions, and adopt the best of the three solution. Finally, run k iterations of Perturbation-SAWMBS.

6. Computational study

All our heuristics were coded in C++ and compiled using Microsoft Visual Studio 2008 in Win32 mode. All tests were performed on a computer with an Intel Core2 Quad CPU Q8200 2.33 GHz. Algorithms were run as a single process/thread.

6.1. Benchmark problem instances

The algorithms were tested on five classes of benchmark problem instances, all of which can be downloaded from the web page of EURO Special Interest Group on Cutting and Packing (ESICUP) (<http://paginas.fe.up.pt/~esicup/>). The first two, the u class and the t class, were developed by Falkenauer (1996). The u class has item weights drawn from an integer uniform distribution on $[20, 100]$ and bin capacity $c = 150$. There are four sets in this class, namely u_{120} , u_{250} , u_{500} and u_{1000} ; each consisting of 20 instances with $n = 120, 250, 500$ and 1000 items, respectively. The t class has item weights drawn from a uniform distribution on $[25, 50]$ and $c = 100$. Item weights in this class are real numbers with 0.1 resolution. Therefore, the weights and the bin capacity had to be multiplied by 10 to be converted to integer numbers. There are also four sets in this class, namely t_{60} , t_{120} , t_{249} and t_{501} ; each consisting of 20 instances with $n = 60, 120, 249$ and 501 items, respectively. The t class is considered difficult, because in an optimal solution of each instance, each bin contains 3 items with zero slack (hence the name 'triplets class'). All problem instances in both the u and t classes have been solved to optimality with the exact algorithm of Valério de Carvalho (2002).

A third class of benchmark problem instances, developed by Scholl et al. (1997), consists of set_1, set_2, and set_3. Set_1 has 720 instances with $c = 100, 120, 150, n = 50, 100, 200, 500$, and item weights drawn from an integer uniform distribution on $[1, 100], [20, 100]$, and $[30, 100]$. Set_2 has 480 instances with $c = 1000, n = 50, 100, 200, 500$ and item weights such that each bin has on average 3 to 9 items. Set_3 has 10 instances with $c = 100,000, n = 200$, and item weights drawn from a uniform distribution on $[20,000, 35,000]$. Scholl et al. (1997) found optimal solutions for 1184 instances in this class. For the remaining 26 instances, optimal solutions were found by Alvim et al. (2004).

A fourth class of benchmark problem instances, developed by Schwerin and Wäscher (1997), contains two sets, was_1 and was_2. Each set has 100 instances with $c = 1000$ and item weights from $[150, 200]$. Was_1 has $n = 100$ items in each instance, while was_2 has $n = 120$ items. For all instances in this class, optimal solutions are known.

A fifth class of benchmark problem instances, developed by Wäscher and Gau (1996), is called gau_1 and contains 17 problem instances with $c = 10,000$ and various values of n and item weights. For 13 of these instances, optimal solutions were previously

known. For one of them, a solution better than the previously best-known has been found in this study (see Section 6.4 for details) and is used in the report below. For the remaining three instances, the optimality gap is one bin.

6.2. SAW heuristics and reductions

Table 1 shows a comparison of the results of the heuristics defined in Sections 2 and 3. For each, results obtained with help of the reductions defined in Section 4 are reported in the left part of the table, while the results obtained without using the reductions are presented in the right part of the table. The column in the middle shows the impact of adding reductions to each heuristic in terms of the number of solutions improved (+) and worsened (–). The quality of results found by each heuristic is presented in terms of the number of best-known solutions found (Best), the average and maximum absolute deviation from best-known solutions (Dev. Avg. and Max.), and the average and maximum percentage deviation from best-known solutions (%Dev. Avg. and Max.). The average and maximum processing times are reported in milliseconds.

The impact of the reductions on most non-SAW heuristics is negligible. One FFD solution is changed (improved) and none of the B2F and MBS' solutions is changed. Average processing times of these three heuristic are unaffected, while maximum processing times are roughly doubled. However, the impact on the MBS solutions is marvelous: 426 (out of 1587) solutions are improved, none is worsened, and the average deviation is reduced from 1.75% to 0.99%. This improvement is achieved without increasing the average and maximum computation times.

For SAW-heuristics, some solutions are improved by adding the reductions, while others are worsened. However, for all three SAW-heuristic, more solutions are improved than worsened. Also, average deviations are consistently better when reductions are used compared with when they are not. Adding reductions to SAWB2F and SAWMBS' has no effect on average computation times and roughly doubles maximum computation times. For SAWMBS, adding reductions reduces both the average and the maximum computation times by about 50%.

Overall, reductions have a positive impact on the performance of all heuristics, especially on MBS and SAWMBS.

The SAW principle brings significant benefits to all three BOHs without incurring any costs:

- SAWB2F solutions are on average better than B2F solutions, especially when reductions are used: the average deviation for SAWB2F is 0.93% versus 1.04% for B2F. Applying the SAW principle to B2F has no effect on average and maximum computation times.
- For MBS and MBS', the impact of adding the SAW principle is immensely positive on both solutions and processing times. Consider just the case with reductions: average deviations are reduced 3–4 times, from 0.99% and 1.05% to 0.27% and 0.30%,

respectively. At the same time, average processing times are reduced by 3–5.5 times and maximum processing times are reduced by an order of magnitude. Although both SAWMBS and SAWMBS' with reductions have exponential computational complexity, in our tests they perform roughly as fast as the polynomial FFD, B2F, and SAWB2F with reductions.

Recall that MBS' differs from MBS in that the former always packs the largest available item in the bin at hand. This gives MBS' an advantage over MBS, both in terms of processing times and solution quality, when both heuristics are used without reductions (Fleszar and Hindi, 2002). However, the relationship changes when reductions are added: MBS' is still roughly twice as fast as MBS, but its results are significantly worse. The advantage of fixing the largest item decreases even more when the SAWMBS' is compared with SAWMBS, both applied with reductions: SAWMBS is only about 20% slower than SAWMBS' on average, while its solutions are significantly better, both in terms of the best-known solutions found (1385 vs. 1372) and the average deviations (0.27% vs. 0.30%). Thus, we conclude that SAWMBS with reductions is the best of the methods tested in this section.

In order to guard against excessively high computation times of SAWMBS, which has an exponential worst-case complexity, a restriction on the number of iterations (recursive calls) within the subset enumeration can be introduced. In our experiments, restricting the number of iterations for each bin to $5n$ did not affect the results of SAWMBS at all, while limiting overall computational complexity to $O(n^3)$.

Note also that SAWB2F with or without reductions has $O(n^2)$ computational complexity. To the best of our knowledge, SAWB2F with reductions is the best-known quadratic algorithm for the BPP.

6.3. Perturbation-SAWMBS heuristic

Recall from Section 5 that Perturbation-SAWMBS is an improvement heuristic, that performs a series of perturbations in an attempt to improve the initial solution. Several values of the number of perturbations, k , were experimented with. With $k = 2,000$, Perturbation-SAWMBS takes on average a modest 8 millisecond per instance and finds the best-known solutions for all but one instance, namely u_250_12, for which the number of bins used is equal to the best-known plus one. This instance could only be solved when k was increased to 50,000, which increased the average time per instance to 178 millisecond.

Table 2 shows the results of Perturbation-SAWMBS with $k = 2,000$ compared with the three best-performing improvement heuristics reported in the literature: the compound heuristic, C_BP, of Singh and Gupta (2007); the hybrid improvement heuristic, HI_BP, of Alvim et al. (2004); and the weight-annealing heuristic, WA, of Loh et al. (2008). The results of C_BP and HI_BP were taken from the respective papers and adjusted taking into account two new best-known solutions for instances in set gau_1, one first

Table 1
Comparison of BOHs with and without reductions over 1587 instances.

Heuristic	With reductions								Reductions impact +/-	Without reductions							
	Best	Dev.		%Dev.		Time (millisecond)		Best		Dev.		%Dev.		Time (millisecond)			
		Avg.	Max.	Avg.	Max.	Avg.	Max.			Avg.	Max.	Avg.	Max.	Avg.	Max.		
FFD	792	1.49	24	2.69	20.0	.08	1.7	1/0	791	1.49	24	2.69	20.0	.08	0.8		
B2F	1120	.41	9	1.04	14.3	.09	1.7	0/0	1120	.41	9	1.04	14.3	.09	0.9		
MBS	1104	.36	5	.99	14.3	.67	30.8	426/0	725	.97	9	1.75	16.7	.69	30.8		
MBS'	1104	.42	9	1.05	14.3	.31	15.6	0/0	1104	.42	9	1.05	14.3	.30	16.3		
SAWB2F	1158	.34	5	.93	14.3	.09	1.7	77/16	1106	.39	5	.97	14.3	.09	0.9		
SAWMBS	1385	.16	4	.27	14.3	.12	1.7	24/3	1368	.17	4	.28	14.3	.22	3.4		
SAWMBS'	1372	.18	4	.30	14.3	.10	1.7	60/19	1335	.21	4	.33	14.3	.10	0.8		

reported by Loh et al. (2008) and one found by us (see Section 6.4). The results of the WA were obtained by us using the C code kindly provided by Dr. Bruce Golden (compiled with full optimizations in Microsoft Visual Studio 2008). We are aware of the discrepancy between the previously published results of WA and the results of WA presented here, but we believe that the latter are correct.

The quality of solutions is compared in Table 2 based on the number of instances for which each algorithm fails to find best-known solutions. Note that in all such cases, all compared algorithms found solutions with the number of bins equal to the best-known plus one. Processing times, average and maximum, are now presented in seconds. For each algorithm, the processor used to obtain the results is shown at the bottom of the table.

Perturbation-SAWMBS finds solutions that are on average slightly better than those of C_BP and HI_BP and much better than those of WA. It fails to find the best-known solution for only one instance, while C_BP and HI_BP fail for six and seven instances, respectively. Interestingly, both Perturbation-SAWMBS and C_BP fail to find the optimal solution for the same instance in set u_250. Perturbation-SAWMBS is also significantly faster than the other algorithms. Assuming that the processors on which C_BP and HI_BP were tested are about 2–2.5 times slower than the processor used in this study, our algorithm is still at least five times faster than any of the other presented algorithms. Note that C_BP of Singh and Gupta (2007) uses the best-known lower bound for early termination, which gives it an additional advantage in the processing time comparison with the other algorithms, including ours, which use internally computed, often weaker than the best-known, lower bounds.

Similarly to Alvim et al. (2004) and Singh and Gupta (2007), we performed a robustness test by running the Perturbation-SAWMBS five times. Overall, our algorithm missed the best-known solutions in only 13 cases, while HI_BP of Alvim et al. (2004) and C_BP of Singh and Gupta (2007) missed them in 39 and 50 cases (numbers taken from respective papers and adjusted taking into account two of the subsequently improved best-known solutions). Broadly, all three algorithms appear to be of similar robustness.

6.4. New best-known solutions

All benchmark problem instances used in this study have been previously solved to optimality with the exception of some

instances from set gau_1. Instances in this set proved to be more difficult than in the other sets. Recently, Alvim et al. (2004), Singh and Gupta (2007), and Loh et al. (2008) reported improved solutions for some of these instances, leaving only four instances, for which best-known solutions were not proven to be optimal. Perturbation-SAWMBS with $k = 2,000$ found a solution better than the best-known for one of the four instances: for TEST0030, a solution using 27 instead of 28 bins was found. For all other instances in gau_1, solutions using the same number of bins as previously best-known were found. The list of currently best-known lower bounds (column LB) and solution values (column Bins) for all 17 instances of set gau_1 are shown in Table 3.

6.5. Additional hard problem instances

Following a recommendation of a reviewer, we performed one more test on the data set of difficult problem instances, called hard28, used for example by Belov and Scheithauer (2006), for which our algorithms were not specifically designed. This set has 28 instances with $n \in \{160, 180, 200\}$, $c = 1000$, and items weights drawn from $[1, 800]$. The results are presented in Table 4 in the same format as in Table 1. The simplest heuristic, FFD, finds optimal solutions for five instances and solutions worse than optimal by one bin for all the remaining instances. In all solutions, the number of bins used is equal to the linear relaxation lower bound plus one. None of the other heuristics is able to improve these solutions; in fact, B2F, MBS, MBS', and SAWMBS find worse solutions for some instances, and SAWB2F, SAWMBS', Perturbation-SAWMBS, and WA find the same solutions as FFD. This shows that in

Table 3

Best-known lower bounds (LB) and solutions values (Bins) for all instances of set gau_1.

ID	Name	LB		Bins	ID	Name	LB		Bins
1	TEST0022	14	<	15	10	TEST0014	23	<	24
2	TEST0065	15	<	16	11	TEST0082	24		24
3	TEST0097	12		12	12	TEST0044	14		14
4	TEST0058	20		20	13	TEST0030	27		27 ^a
5	TEST0055	15		15	14	TEST0005	28		28
6	TEST0049	11		11	15	TEST0095	16		16
7	TEST0075	13		13	16	TEST0055	20		20
8	TEST0054	14		14	17	TEST0084	16		16
9	TEST0068	12		12					

^a New best-known solution found by Perturbation-SAWMBS.

Table 2

Comparison of Perturbation-SAWMBS with other heuristics.

Set	Inst.	Pert.-SAWMBS			C_BP			HI_BP			WA		
		Not	Time (second)		Not	Time (second)		Not	Time (second)		Not	Time (second)	
		Best	Avg.	Max.	Best	Avg.	Max.	Best	Avg.	Max.	Best	Avg.	Max.
u_120	20	0	.00	.00	0	.01	.05	0	.00	.01	2	.01	.03
u_250	20	1	.01	.06	1	.04	3.85	0	.12	2.20	4	.05	.15
u_500	20	0	.00	.00	0	.05	.87	0	.00	.01	2	.21	.81
u_1000	20	0	.00	.00	0	.03	4.71	0	.01	.02	1	.88	5.08
t_60	20	0	.00	.01	0	.00	.02	0	.37	1.65	20	.01	.01
t_120	20	0	.00	.01	0	.01	.02	0	.85	4.53	20	.02	.03
t_249	20	0	.00	.01	0	.01	.02	0	.22	.51	20	.11	.11
t_501	20	0	.01	.02	0	.02	.05	0	2.49	19.26	20	.48	.49
set_1	720	0	.01	.14	1	.31	13.98	0	.19	19.23	17	.25	1.92
set_2	480	0	.00	.11	0	.04	2.66	0	.01	1.19	12	.04	.89
set_3	10	0	.16	.37	0	.92	2.70	0	4.60	44.76	1	.08	.09
was_1	100	0	.00	.00	0	.00	.02	0	.02	.14	0	.00	.01
was_2	100	0	.01	.04	0	.00	.05	0	.01	.26	0	.02	.06
gau_1	17	0	.04	.20	4	.43	2.14	7	.62	2.06	4	.02	.13
All	1587	1	.01	.37	6	.17	13.98	7	.18	44.76	123	.15	5.08
		Core2 2.33 GHz			P4 2.4 GHz			P4 1.7 GHz			Core2 2.33 GHz		

Table 4
Comparison over hard28 data set.

Heuristic	With reductions							Red. impact +/-	Without reductions						
	Best	Dev.		%Dev.		Time (millisecond)			Best	Dev.		%Dev.		Time (millisecond)	
		Avg.	Max.	Avg.	Max.	Avg.	Max.			Avg.	Max.	Avg.	Max.	Avg.	Max.
FFD	5	.82	1	1.20	1.7	.07	.08	0/0	5	.82	1	1.20	1.7	.05	.06
B2F	4	1.07	2	1.56	3.3	.08	.13	0/0	4	1.07	2	1.56	3.3	.07	.08
MBS	3	1.29	4	1.86	4.9	.11	.15	21/0	1	3.32	10	4.62	12.4	.15	.23
MBS'	2	1.39	3	2.01	5.0	.08	.13	0/0	2	1.39	3	2.01	5.0	.07	.09
SAWB2F	5	.82	1	1.20	1.7	.08	.09	0/0	5	.82	1	1.20	1.7	.07	.08
SAWMBS	4	1.11	3	1.60	4.5	1.19	4.64	0/0	4	1.11	3	1.60	4.5	1.53	5.60
SAWMBS'	5	.82	1	1.20	1.7	.13	.21	0/0	5	.82	1	1.20	1.7	.12	.25
Pert.-SAWMBS	5	.82	1	1.20	1.7	248.08	482.57								
WA									5	.82	1	1.20	1.7	59.00	74.91

practice taking the best solution of several heuristics, including simple ones, is always the safest approach.

Several additional remarks are noteworthy. MBS performs extremely badly on set hard28, compared to all other heuristics. Adding reductions has no impact on any of the heuristics except for MBS, for which it improves 21 out of 28 solutions. Adding the SAW-principle to B2F, MBS, and MBS' improves solution quality of all three heuristics. However, this time the SAW principle increases computation time, especially in the case of SAWMBS. As a result, the Perturbation-SAWMBS, which repeatedly invokes SAWMBS, is slower than WA on this data set. Finally, the FFD solutions cannot be improved even if the number of iterations within Perturbation-SAWMBS is increased to $k = 100,000$. Clearly, the instances are very difficult and can be used in further work on heuristics for BPP.

7. Conclusions and further research

The contributions of this work are:

- The SAW principle proposed here proved both effective and efficient. Its effectiveness is due to the embedded lookahead mechanism that guards against using too many small items early on in bin-oriented heuristics, while its efficiency is due to the fact that it prunes the search tree effectively in those heuristics that consider a multiplicity of subsets.
- Applying the proposed simple reduction after packing each bin has a minimal effect on computation times while improving significantly solutions of some bin-oriented heuristics.
- The three bin-oriented heuristics based on the SAW principle, augmented with reduction methods, dominate all known simple heuristics in terms of solution quality, while remaining competitive in terms of computation time.
- The Perturbation-SAWMBS improvement heuristic outperforms the best of the known improvement heuristics in terms of solution quality as well as in terms of computation time. Furthermore, a new solution better than the best known was found for one benchmark instance.

Several further research directions following this work could be outlined. First, it would be interesting to investigate how the SAW principle affects the asymptotic-worst-case performance of the underlying bin-oriented heuristics. While the simple FFD heuristic has the asymptotic worst-case performance ratio of $11/9 = 1.2222$ (Johnson et al., 1974), the ratio for MBS was recently shown by Caprara and Pferschy (2004) to be much worse, between 1.6067 and 1.6210, and for MBS' by Caprara and Pferschy (2005) to be between 1.30335 and $1.4444 = 13/9$. It would be interesting to investigate the asymptotic-worst-case performance when the SAW principle is employed.

Secondly, the usefulness of the SAW principle could be investigated for a semi-online version of the BPP, in which a new

set of items arrives periodically, for example in a cutting stock industry. While in such cases order priorities or due dates play a major role, avoidance of using too many small items early on in the process, which is facilitated by the SAW principle, could be beneficial.

Finally, it may be worthwhile investigating effectiveness of a principle similar to the one presented here for two- and more-dimensional bin-packing problems.

Acknowledgements

We would like to express our gratitude to Dr. Bruce Golden for providing us with the source code of the weight-annealing (WA) algorithm.

We are also grateful for the comments and queries of the referees that helped us improve content and presentation.

Appendix A

Table 5 provides the notation used throughout the paper. Algorithms 1 and 2 present the pseudo-codes of the recursive subset-sum (SS) procedures, MBSSS and SAWMBSSS, used within the MBS and SAWMBS heuristics, respectively, to find the subset that should be packed in one bin. It is assumed that before an SS procedure is invoked, all unpacked items are renumbered such that their weights are non-increasing and are stored in list L . Initially, L contains all available items. Before the appropriate procedure is invoked, global variables are initiated: the subset of selected items, A , is set to $\emptyset$ and the incumbent, A^* , is set to the FFD solution for one bin. MBS calls MBSSS and SAWMBS calls SAWMBSSS, both

Table 5
Notation used in the paper.

Symbol	Description
c	bin capacity
N	set of all items, $N = \{1, \dots, n\}$
w_i	weight of item i
s_i	slack in the bin in which item i is packed
L	list of currently unpacked items renumbered such that their weights are non-increasing, $L = (1, \dots, L)$
A	feasible subset of currently selected items from L
A^*	incumbent subset
$w(A)$	total weight of items in A , $w(A) = \sum_{i \in A} w_i$
$\bar{w}(A)$	average weight of items in A , $\bar{w}(A) = w(A) / A $
$s(A)$	slack remaining in a bin after packing A in it, $s(A) = c - w(A)$
j	item currently considered for addition to A , $j \in L$
$p_{\min}$	minimum number of items that must be added to A for it to be maximal
$p_{\max}$	maximum number of items that can be added to A for it to improve the incumbent
$\bar{w}_{\max}$	maximum average weight if at least $p_{\min}$ items are added to A
$s_{\min}$	minimum slack if at most $p_{\max}$ items are added to A

with parameter $j = 1$. MBS' and SAWMBS' differ from MBS and SAWMBS in that they fix the first item in the subset of selected items by setting $A = \{1\}$ and invoking the appropriate SS procedure with $j = 2$. When the SS enumeration is completed, a new bin is added, and the items of A^* are assigned to it and removed from list

L . The processing is repeated until L becomes empty. Note that each line in Algorithms 1 and 2 has a $O(1)$ computational complexity, since all values of $w(\cdot)$, $\bar{w}(\cdot)$, $s(\cdot)$ can be computed incrementally and subset maximality can be checked in $O(1)$ provided that the weight of the smallest omitted item is updated incrementally.

Algorithm 1. Procedure MBSSS

```

Procedure MBSSS( $j$ )
  if  $j > |L|$  or  $w_{|L|} > s(A)$  then
    /* No items can be added to  $A$ , update the incumbent and backtrack */
    if  $s(A) < s(A^*)$  then  $A^* \leftarrow A$ ;
    return;
  end
  while  $w_j > s(A)$  do  $j \leftarrow j + 1$  /* skip too large items */
  while  $j \leq |L|$  do
    /* Reductions */
    if  $s(A) - w(\{j, \dots, |L|\}) \geq s(A^*)$  then return;
    /* Add  $j$  to  $A$ , invoke a recursive call, and then remove  $j$  from  $A$  */
     $A \leftarrow A \cup \{j\}$ ;
    MBSSS( $j + 1$ );
     $A \leftarrow A \setminus \{j\}$ ;
    if  $s(A^*) = 0$  then return; /* backtrack if incumbent optimal */
     $j \leftarrow j + 1$ ;
    while  $j \leq |L|$  and  $w_j = w_{j-1}$  do  $j \leftarrow j + 1$  /* skip items of the same weight */
  end
end procedure

```

Algorithm 2. Procedure SAWMBSSS

```

procedure SAWMBSSS( $j$ )
  if  $j > |L|$  or  $w_{|L|} > s(A)$  then
    /* No items can be added to  $A$ , update the incumbent and backtrack */
    if  $A$  is not maximal then return;
    if  $\bar{w}(A^*) \geq \bar{w}(L)$  then /* if incumbent has sufficient average weight */
      if  $\bar{w}(A) \geq \bar{w}(L)$  and  $s(A) < s(A^*)$  then  $A^* \leftarrow A$ ;
    end
    else if  $\bar{w}(A) > \bar{w}(A^*)$  or ( $\bar{w}(A) = \bar{w}(A^*)$  and  $s(A) < s(A^*)$ ) then  $A^* \leftarrow A$ ;
    return;
  end
  while  $w_j > s(A)$  do  $j \leftarrow j + 1$  /* skip too large items */
  while  $j \leq |L|$  do
    /* Reductions */
    if  $\bar{w}(A^*) \geq \bar{w}(L)$  and  $s(A) - w(\{j, \dots, |L|\}) \geq s(A^*)$  then return;
    if subset  $A \cup \{j, \dots, |L|\}$  would be feasible but not maximal then return;
    /* Additional SAW-related reductions, see Section 3 for details */
     $p_{\min} \leftarrow \min\{[s(A)/w_j], |L| - j + 1\}$ ;
     $\bar{w}_{\max} \leftarrow (w(A) + p_{\min} w_j) / (|A| + p_{\min})$ ;
     $p_{\max} \leftarrow \lfloor c / \min\{\bar{w}(L), \bar{w}(A^*)\} \rfloor - |A|$ ;
     $s_{\min} \leftarrow \max\{s(A) - p_{\max} w_j, 0\}$ ;
    if  $p_{\min} > p_{\max}$  then return;
    if  $\bar{w}(A^*) \geq \bar{w}(L)$  then /* if incumbent has sufficient average weight */
      if  $\bar{w}_{\max} < \bar{w}(L)$  or ( $\bar{w}_{\max} \geq \bar{w}(L)$  and  $s_{\min} \geq s(A^*)$ ) then return;
    end
    else if  $\bar{w}_{\max} < \bar{w}(A^*)$  or ( $\bar{w}_{\max} = \bar{w}(A^*)$  and  $s_{\min} \geq s(A^*)$ ) then return;
    /* Add  $j$  to  $A$ , invoke a recursive call, and then remove  $j$  from  $A$  */
     $A \leftarrow A \cup \{j\}$ ;
    SAWMBSSS( $j + 1$ );
     $A \leftarrow A \setminus \{j\}$ ;
    if  $\bar{w}(A^*) \geq \bar{w}(L)$  and  $s(A^*) = 0$  then return; /* backtrack if incumbent optimal */
     $j \leftarrow j + 1$ ;
    While  $j \leq |L|$  and  $w_j = w_{j-1}$  do  $j \leftarrow j + 1$  /* skip items of the same weight */
  end
end procedure

```


References

- Alvim, A.C.F., Ribeiro, C.C., Glover, F., Aloise, D.J., 2004. A hybrid improvement heuristic for the one-dimensional bin packing problem. *Journal of Heuristics* 10 (2), 205–229.
- Belov, G., Scheithauer, G., 2006. A branch-and-cut-and-price algorithm for one-dimensional stock cutting and two-dimensional two-stage cutting. *European Journal of Operational Research* 171 (1), 85–106.
- Bhatia, A.K., Basu, S.K., 2004. Packing bins using multi-chromosomal genetic representation and better fit heuristic. *Lecture Notes in Computer Science* 3316, 181–186.
- Caprara, A., Pferschy, U., 2004. Worst-case analysis of the subset sum algorithm for bin packing. *Operations Research Letters* 32 (2), 159–166.
- Caprara, A., Pferschy, U., 2005. Modified subset sum heuristics for bin packing. *Information Processing Letters* 96 (1), 18–23.
- Coffman, E.G., Garey, M.R., Johnson, D.S., 1997. Approximation algorithm for bin packing: A survey. In: Hochbaum, D. (Ed.), *Approximation Algorithm for NP-hard Problems*. PWS Publishing, pp. 46–93.
- Crainic, T., Perboli, G., Pezzuto, M., Tadei, R., 2007a. Computing the asymptotic worst-case of bin packing lower bounds. *European Journal of Operational Research* 183 (3), 1295–1303.
- Crainic, T., Perboli, G., Pezzuto, M., Tadei, R., 2007b. New bin packing fast lower bounds. *Computers and Operations Research* 34 (11), 3439–3457.
- Falkenauer, E., 1996. A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics* 2 (1), 5–30.
- Fekete, S., Schepers, J., 2001. New classes of fast lower bounds for bin packing problems. *Mathematical Programming* 91 (1), 11–31.
- Fleszar, K., Hindi, K.S., 2002. New heuristics for one-dimensional bin-packing. *Computers and Operations Research* 29 (7), 821–839.
- Friesen, D., Langston, M., 1991. Analysis of a compound bin packing algorithm. *SIAM Journal on Discrete Mathematics* 4, 61.
- Garey, M.R., Johnson, D.S., 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- Gupta, J.N.D., Ho, J.C., 1999. A new heuristic algorithm for the one-dimensional bin-packing problem. *Production Planning and Control* 10 (6), 598–603.
- Johnson, D., Demers, A., Ullman, J., Garey, M., Graham, R., 1974. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing* 3 (4), 299–325.
- Loh, K.-H., Golden, B., Wasil, E., 2008. Solving the one-dimensional bin packing problem with a weight annealing heuristic. *Computers and Operations Research* 35 (7), 2283–2291.
- Martello, S., Toth, P., 1990. *Knapsack Problems*. Wiley, available online on <<http://www.or.deis.unibo.it/knapsack.html>>.
- Poli, R., Woodward, J., Burke, E.K., 2007. A histogram-matching approach to the evolution of bin-packing strategies. In: *IEEE Congress on Evolutionary Computation 2007 (CEC 2007)*, pp. 3500–3507.
- Rohlfshagen, P., Bullinaria, J., 2007. A genetic algorithm with exon shuffling crossover for hard bin packing problems. *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation*, 1365–1371.
- Scholl, A., Klein, R., Jürgens, C., 1997. BISON: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Computers and Operations Research* 24 (7), 627–645.
- Schwerin, P., Wäscher, G., 1997. The bin-packing problem: A problem generator and some numerical experiments with FFD packing and MTP. *International Transactions in Operational Research* 4 (5/6), 377–389.
- Singh, A., Gupta, A.K., 2007. Two heuristics for the one-dimensional bin-packing problem. *OR Spectrum* 29 (4), 765–781.
- Valério de Carvalho, J.M., 1999. Exact solution of bin-packing problems using column generation and branch-and-bound. *Annals of Operations Research* 86, 629–659.
- Valério de Carvalho, J.M., 2002. LP models for bin packing and cutting stock problems. *European Journal of Operational Research* 141 (2), 253–273.
- Vanderbeck, F., 1999. Computational study of a column generation algorithm for bin packing and cutting stock problems. *Mathematical Programming* 86 (3), 565–594.
- Wäscher, G., Gau, T., 1996. Heuristics for the integer one-dimensional cutting stock problem: A computational study. *OR Spectrum* 18 (3), 131–144.